

```
$ javac ArrayVsList.java && java ArrayVsList
```

Array NO GLYPH List

// 陣列 vs 列表 - 陣列與列表

```
int[] arr → 陣列
```

```
List<T> → 列表
```

\$ cat agenda.md



1. Java — Array List □□□□

2. Array □□□□

3. Array □□□□

4. List □□□□

5. List □□□□

6. □□□□□□

7. Array vs List

8. □□□□

9. Array □□□□□

10. □□□□

11. □□□□

12. Array — □□□□□□□□

13. □□□□□□□□

PART 01

Array□□□□

□□□□□□□

Array □□□□

```
1 // 000000000000
2 int[] numbers1 = new int[5];
3 int[] numbers2 = {1, 2, 3, 4, 5};
4 int[] numbers3 = new int[]{1, 2, 3, 4, 5};
5
6 // 0000
7 numbers1[0] = 10;
8 numbers1[1] = 20;
9 System.out.println(numbers1[0]); // 10
10
11 // 0000
12 System.out.println(numbers2.length); // 5
```

0000
000000

0000
00000

0000
array[index]

000
00000

PART 02

List□□□□

□□□□□□□□

List □□□□

```
1 import java.util.ArrayList;
2 import java.util.List;
3
4 List<Integer> numbers = new ArrayList<>();
5 List<String> names = new ArrayList<>();
6
7 numbers.add(10);
8 numbers.add(20);
9 names.add("Alice");
10
11 System.out.println(numbers.get(0)); // 10
12 numbers.set(0, 100);
13 numbers.remove(1);
14 System.out.println(numbers.size());
```

□□□□
□□□□

□□□□
add/remove/contains

□□□□
List<T>

□□□□
□□□□□

Array vs List

□□	Array	List
□□	□□□□	□□□□
□□	<code>array[index]</code>	<code>list.get(index)</code>
□□	✗ NO INDEX NO INDEX NO INDEX	<code>list.add()</code>
□□	✗ NO INDEX NO INDEX NO INDEX	<code>list.remove()</code>
□□	<code>array.length</code>	<code>list.size()</code>
□□□□	✓ NO INDEX NO INDEX NO INDEX NO INDEX	□□□□□

Array 50

List 100

[0]

10

[1]

20

[2]

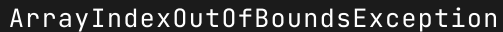
—

[3]

—

[4]

—

[illegible]

□ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □




[10.1.1.1](#)
[10.1.1.2](#)
[10.1.1.3](#)
[10.1.1.4](#)
[10.1.1.5](#)
[10.1.1.6](#)
[10.1.1.7](#)
[10.1.1.8](#)
[10.1.1.9](#)
[10.1.1.10](#)
[10.1.1.11](#)
[10.1.1.12](#)
[10.1.1.13](#)
[10.1.1.14](#)
[10.1.1.15](#)
[10.1.1.16](#)
[10.1.1.17](#)
[10.1.1.18](#)
[10.1.1.19](#)
[10.1.1.20](#)
[10.1.1.21](#)
[10.1.1.22](#)
[10.1.1.23](#)
[10.1.1.24](#)
[10.1.1.25](#)
[10.1.1.26](#)
[10.1.1.27](#)
[10.1.1.28](#)
[10.1.1.29](#)
[10.1.1.30](#)
[10.1.1.31](#)

Array — □□□□□□□□

```
1 int[] intArray = new int[3];
2 intArray[0] = 10; // 000 int
3
4 String[] strArray = new String[3];
5 strArray[0] = "Hello";
```



List — □□□□□□

```
1 List<Integer> intList = new ArrayList<>();
2 intList.add(10); // Autoboxing: int → Integer
3
4 int value = intList.get(0); // Unboxing
```



List □□□

```
1 List<String> strList = new ArrayList<>();
2 List<Integer> intList = new ArrayList<>();
3
4 // □□□□□□□□
5 strList.getClass() == intList.getClass(); // true
```

□□□□□□□□□□□□□□□□

Array □□□□

```
1 String[] strArray = new String[5];
2 Integer[] intArray = new Integer[5];
3
4 strArray.getClass() == intArray.getClass(); // false
```

Array □□□□□□□□□□□□□□

Array □□□



ArrayList
LinkedList
Vector
Stack
Deque
PriorityQueue
Queue
ConcurrentQueue
ConcurrentLinkedQueue
ConcurrentSkipListSet
ConcurrentSkipListMap
ConcurrentHashMap



ArrayList
LinkedList
Vector
Stack
Deque
PriorityQueue
Queue
ConcurrentQueue
ConcurrentLinkedQueue
ConcurrentSkipListSet
ConcurrentSkipListMap
ConcurrentHashMap



ArrayList
LinkedList
Vector
Stack
Deque
PriorityQueue
Queue
ConcurrentQueue
ConcurrentLinkedQueue
ConcurrentSkipListSet
ConcurrentSkipListMap
ConcurrentHashMap
ConcurrentMap
ConcurrentMap



ArrayList
LinkedList
Vector
Stack
Deque
PriorityQueue
Queue
ConcurrentQueue
ConcurrentLinkedQueue
ConcurrentSkipListSet
ConcurrentSkipListMap
ConcurrentHashMap

```
1 // 3x3 □□
2 char[][] board = new char[3][3];
3 board[1][1] = 'X';
```

List □□□



ArrayList
LinkedList
Vector
Stack
Deque
PriorityQueue
Queue
ConcurrentQueue
ConcurrentLinkedQueue
ConcurrentSkipListSet
ConcurrentSkipListMap
ConcurrentHashMap



ArrayList
LinkedList
Vector
Stack
Deque
PriorityQueue
Queue
ConcurrentQueue
ConcurrentLinkedQueue
ConcurrentSkipListSet
ConcurrentSkipListMap
ConcurrentHashMap



ArrayList
LinkedList
Vector
Stack
Deque
PriorityQueue
Queue
ConcurrentQueue
ConcurrentLinkedQueue
ConcurrentSkipListSet
ConcurrentSkipListMap
ConcurrentHashMap



ArrayList
LinkedList
Vector
Stack
Deque
PriorityQueue
Queue
ConcurrentQueue
ConcurrentLinkedQueue
ConcurrentSkipListSet
ConcurrentSkipListMap
ConcurrentHashMap

```
1 List<String> users = new ArrayList<>();
2 users.add("user1");
3 users.remove("user1");
```



Array — □□□□□□ List — □□□□□□

```
1  int[] examScores = new int[4];
2  examScores[0] = 85;
3  examScores[1] = 92;
4  examScores[2] = 78;
5  examScores[3] = 96;
6
7  int total = 0;
8  for (int score : examScores) {
9      total += score;
10 }
11 double avg = (double) total / examScores.length;
```



Array

- □□□□□□□□□□
- □□□□□□□
- array[index][].length
- □□□□□□□□□□

List

- □□□□□□□□□□
- □□□□□□□Autoboxing□
- get/set/add/remove[].size()
- □□□□□□□□□□ API

□□□□□□□□□□□□□□□□ — □□□□□□□□□□□□□□□□□□□□



□□□□□Java Stream□HashSet/TreeSet

```
$ java --collections-framework
```